

PHC Pulse — Technical Architecture

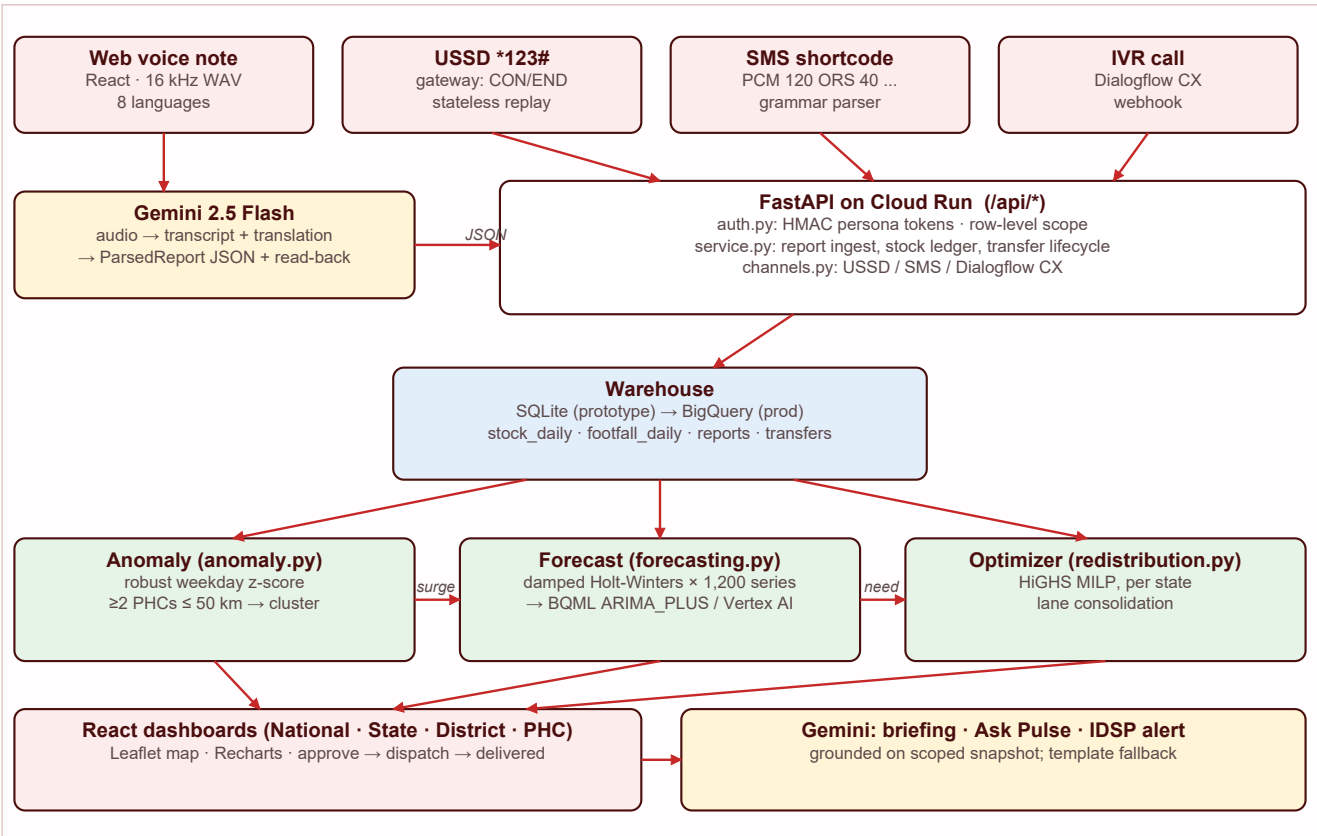
Filled · HOW is it built?

Purpose. Define HOW the product is implemented: stack, services, data flow, integrations, security and deployment.

Filled from the repository PHC-Pulse-GDG-Code-for-community (main @ fd42d41) on 26 Sep 2026. Numbers quoted were measured by running the engine and the 13-test pytest suite (all passing) with PHC_TODAY=2026-09-25, seed 42.

Status legend: **[IMPLEMENTED]** in the repo and tested **[PARTIAL]** partly built **[PLANNED]** designed here, not yet in code

1. Architecture Diagram



engine.py orchestrates it all: it loads the warehouse into numpy arrays, runs anomaly → forecast (with surge) → stock projection → MILP, and caches one read model. Any write (report, transfer) invalidates the cache and the next request recomputes, about 0.7 s for 120 PHCs × 10 drugs × 180 days.

2. Technology Decisions

Layer	Chosen	Alternative	Reason
Frontend	React 19 + Vite 8 SPA, react-router 7, Leaflet / react-leaflet, Recharts; oxlint	Next.js, Angular, Flutter web	No SSR needed. The static build is served by the same container. Leaflet is free and needs no API key. Mobile drawer layout under 820 px.
Backend	Python 3.12, FastAPI 0.136, Pydantic 2, uvicorn	Node/Express, Django	The numerical stack (numpy, scipy HiGHS) runs in-process. Pydantic models double as Gemini response schemas. OpenAPI docs are generated at /docs.
Database	SQLite (WAL) in the prototype; BigQuery warehouse in production (partitioned by day, clustered by state/PHC/drug, row access policies)	Cloud SQL Postgres, Firestore	Zero-ops single-instance demo. Same table shapes as bigquery/schema.sql, so the engine is store-agnostic. BigQuery gives national scale and row-level security.
AI/ML	numpy damped Holt-Winters; robust z-score; scipy.optimize.milp (HiGHS). Production: BQML ARIMA_PLUS, Vertex AI AutoML Forecasting	Prophet, LightGBM, OR-Tools CP-SAT	1,200 series fitted in milliseconds with no training credits. Explainable. A leakage-free walk-forward backtest comes out of the same pass. HiGHS is open source and fast for this MILP size.

Layer	Chosen	Alternative	Reason
Voice/STT	Gemini 2.5 Flash multimodal (audio in, JSON out). Fallback: browser Web Speech API + rule parser (nlu.py)	Cloud Speech-to-Text + separate NLU/translation	One call does STT, translation and extraction for code-switched speech in 8 languages. Response schema means no free-text parsing.
Cloud	Google Cloud: Cloud Run (asia-south1), Cloud Build, Secret Manager, Vertex AI, BigQuery, Dialogflow CX	AWS / Azure	Native access to Gemini, Vertex AI and Dialogflow CX; data stays in the Mumbai region. Deployed by one script.

3. End-to-End Data Flow

Voice/USDD → STT → database → forecast → optimizer → approval → transfer

- **Capture:** voice WAV (≤ 8 MB, ≤ 60 s) → `/api/reports/voice`; USSD → `/api/usdd`; SMS → `/api/sms`; IVR → `/api/dialogflow/webhook`. Callers are identified through the *workers* table (phone → PHC).
- **Understand:** Gemini returns *ParsedReport* (stock lines with kind remaining/received, OPD, fever, diarrhoea, respiratory, beds, staff, confirmation, follow-up, confidence). The fallback is `nlu.parse` on text.
- **Persist:** `service.apply_report` upserts today's *stock_daily* row (opening carried forward; dispensed = opening + received + adjustment – remaining) and *footfall_daily* (beds clamped to capacity), inserts a *reports* row, then calls `engine.invalidate()`.
- **Forecast:** Holt-Winters on demand = dispensed + unmet. Anomaly surge raises linked drugs. Stock available = closing + incoming – committed transfers, then days-to-stockout, P(stock-out) and risk tier.
- **Optimise:** deficits (critical/high/watch/stocked-out) and donors (surplus) → per-state MILP → shipments REC-xxxxxxx (deterministic hash id) + escalations.
- **Approve → transfer:** an officer approves (scope-checked; returns 409 if the plan changed) → *transfers* row status *approved* → *in_transit* (donor ledger –qty) → *delivered* (recipient ledger +qty). Every step is appended to `history_json` and dashboards recompute.

4. Modules / APIs

Module	Responsibility	Input	Output
Reporting	Voice, text, USSD, SMS and IVR ingestion into the ledger (<code>gemini.py</code> , <code>nlu.py</code> , <code>channels.py</code> , <code>service.py</code>)	POST <code>/api/reports/voice</code> (multipart audio, <code>phc_id</code> , language, previous), <code>/reports/parse</code> , <code>/reports/submit</code> , <code>/usdd</code> , <code>/sms</code> , <code>/dialogflow/webhook</code>	{report, engine}; {report_id, phc, items}; USSD CON/END text; DF fulfillment_response
Forecasting	Demand forecast, stock projection, risk tier, backtest (<code>forecasting.py</code> , <code>engine.py</code>)	<i>stock_daily</i> , <i>footfall_daily</i> , transfers; config horizon 28 d, window 21 d, supply cycle 30 d	GET <code>/api/stock</code> , <code>/api/alerts</code> , <code>/api/phcs/{id}</code> , <code>/api/model</code> : items[] with <i>daily_forecast</i> , <i>cover_days</i> , <i>days_to_stockout</i> , <i>p_stockout</i> , status
Optimization	Redistribution MILP and escalations (<code>redistribution.py</code>)	deficits[], donors[], road-km matrix, cross_state flag	GET <code>/api/redistribution</code> : shipments[] (lines, distance_km, cost_inr, eta_hours, urgency_days), escalations[], stats
Anomaly	Syndromic anomalies, clusters, IDSP alert drafts (<code>anomaly.py</code> , <code>gemini.py</code>)	<i>footfall_daily</i> (28-day baseline), PHC coordinates	anomalies[], clusters[] in <code>/api/alerts</code> and <code>/api/overview</code> ; POST <code>/api/clusters/{id}/alert</code> → {title, english, local_language, recommended_response}
Transfers	Approval and lifecycle with ledger moves (<code>service.py</code>)	POST <code>/api/redistribution/{rec_id}/approve</code> ; POST <code>/api/transfers/{id}/status {status}</code>	Transfer with lines, history, eta_at. Allowed moves: <i>approved</i> → <i>in_transit</i> / <i>cancelled</i> , <i>in_transit</i> → <i>delivered</i>

5. Integrations

Integration	Purpose	Config	Fallback
Gemini	Voice understanding, briefings, Ask Pulse copilot, IDSP alert drafts	GEMINI_API_KEY (AI Studio, via Secret Manager) or GOOGLE_GENAI_USE_VERTEXAI=true + GOOGLE_CLOUD_PROJECT/LOCATION; GEMINI_MODEL (default gemini-2.5-flash); temperature 0.1	Rule parser + template read-back / follow-up; template briefing and alert; copilot says it is unavailable. Audio-only without Gemini → HTTP 503 with a clear message.

Integration	Purpose	Config	Fallback
Speech-to-Text	Transcribe spoken reports	Gemini multimodal (no separate STT hop). Browser Web Speech API with BCP-47 codes (hi-IN, ta-IN, ...)	Text mode or sample sentences; USSD/SMS/IVR channels
Vertex AI	Production demand forecaster and optional Gemini backend	vertex/train_forecast.py --project --region asia-south1 --dataset phc_pulse; training view with syndromic covariates	In-process Holt-Winters (always on); BQML ARIMA_PLUS in bigquery/forecast.sql
Maps/distance	Road distance, cost and ETA per lane	geo.py haversine × ROAD_FACTOR 1.3; AVG_SPEED 35 km/h; INR 600 fixed + 18/km (+2,500 cross-state)	Planned: Google Routes API matrix, cached per lane; haversine stays as the fallback. Map tiles: Leaflet/OSM
HMIS	Facility identity and monthly reporting compatibility	phcs.nin = HMIS NIN; GET /api/hmis/export.csv?month=YYYY-MM (OPD, fever, diarrhoea, ARI, received, consumed, closing, stock-out days)	Seeded HMIS-like backfill (source='hmis'). Live HMIS pull is planned (state pipeline, Phase 2)

6. Security / Reliability

- **[IMPLEMENTED] Auth & roles:** HMAC-SHA256 signed persona tokens (national, state, district DHO, PHC). A request with no token gets a read-only national view for the public demo. There is no password login in the demo; production would use Google Identity / state SSO.
- **[IMPLEMENTED] Row-level scope:** effective_scope narrows but never widens (tested). Approval requires the donor or recipient in scope; PHC role → 403. BigQuery row access policies by state_code.
- **[IMPLEMENTED] Audit:** reports store channel, language, transcript, parsed JSON, reporter, engine and timestamp. Transfers store approved_by and history_json (status, time, actor). Every AI output is tagged with its engine.
- **[IMPLEMENTED] Validation:** Pydantic request models. Drug codes are whitelisted (Gemini hallucinations dropped). Quantities clamped ≥ 0 , beds $\leq$ capacity, audio ≤ 8 MB. The transfer state machine rejects illegal moves (400). Stale recommendations → 409. USSD submit is idempotent per session.
- **[IMPLEMENTED] Failure handling:** Gemini errors are logged and fall back to rules or templates. The MILP has an 8 s time limit and 1% gap; an infeasible result returns an empty plan with the solver message.
- **[PARTIAL] Encryption:** TLS by Cloud Run; at rest by Google-managed keys; secrets in Secret Manager.
- **[PLANNED] Hardening before pilot:** restrict CORS (currently *); add rate limits (Cloud Armor or middleware); role-gate /api/admin/reset; scope-check /api/transfers/{id}/status and /api/reports/submit; rotate PHC_AUTH_SECRET.

7. Deployment

- **Hosting:** multi-stage Dockerfile (node:22-alpine builds the SPA, python:3.12-slim runs uvicorn on \$PORT 8080 and serves /static). *scripts/deploy.sh* runs gcloud run deploy --source (Cloud Build) in asia-south1 with 1 GiB, 1 vCPU, 120 s timeout, max-instances 1.
- **DB:** SQLite at /tmp/phc_pulse.db, seeded on start-up (MIN_INSTANCES=1 keeps demo state). Production: *scripts/export_bigquery.py* + bigquery/schema.sql, then lift the instance limit.
- **CI/CD:** no pipeline in the repo yet. Recommended: a GitHub Action running pytest (13 tests) and oxlint + vite build on each PR, then Cloud Build deploy on main.
- **Environment variables:** GEMINI_API_KEY | GOOGLE_GENAI_USE_VERTEXAI, GOOGLE_CLOUD_PROJECT, GOOGLE_CLOUD_LOCATION; GEMINI_MODEL; PHC_AUTH_SECRET; PHC_TODAY (pin the demo date); PHC_DB_PATH, PHC_STATIC_DIR; PHC_HISTORY_DAYS (180), PHC_SEED (42).
- **Monitoring:** GET /api/health (Gemini status and date), compute_ms on every overview, Cloud Run logs (Gemini failures logged with stack traces). Add uptime checks and an error-rate alert.